

Widgets – interaktive Eingabeparameter

Widgets sind Eingabefelder, mit denen ein Notebook oder Dashboard mit unterschiedlichen Parametern erneut ausgeführt werden kann, ohne den Code selbst zu ändern. Sie eignen sich besonders gut, um dieselbe Abfrage schnell mit verschiedenen Werten zu testen.

Die vier Widget-Typen

Typ	Beschreibung
Text	Freies Textfeld für beliebige Eingaben
Dropdown	Auswahl aus einer vorgegebenen Liste
Combobox	Kombination aus Dropdown und Texteingabe: Auswahl aus Liste ODER eigener Wert
Multiselect	Mehrfachauswahl aus einer vorgegebenen Liste

Alle Widgets speichern ihren Wert als Text (String) und erscheinen in der Notebook-Werkzeugleiste direkt unter dem Titel.

Widgets über die Oberfläche erstellen

Mit Bearbeitungsrechten kann man über **Edit** → **Add parameter** ein Widget anlegen und konfigurieren: Parametername (für den Code), optionale Beschriftung, Widget-Typ, mögliche Werte (bei Dropdown/Multiselect/Combobox) sowie ein optionaler Standardwert.

Widgets per Code erstellen (dbutils.widgets)

Die Widget-API steht in Python, Scala, R und SQL zur Verfügung. Aufbau: Name, Standardwert, Auswahlliste (außer bei Text), optionale Beschriftung.

```
# Python
dbutils.widgets.dropdown("state", "CA", ["CA", "IL", "MI", "NY", "OR", "VA"])
```

```
-- SQL
CREATE WIDGET DROPDOWN state DEFAULT "CA"
CHOICES SELECT * FROM (VALUES ("CA"), ("IL"), ("MI"), ("NY"), ("OR"), ("VA"))
```

Scala und R nutzen dieselbe Logik mit jeweils sprachtypischer Syntax (`Seq(...)` in Scala, `list(...)` in R).

Widget-Werte lesen und entfernen

- `dbutils.widgets.get("name")` – aktuellen Wert eines Widgets abrufen
- `dbutils.widgets.getAll()` – alle Widget-Werte als Map abrufen
- In SQL: `SELECT :widget_name` als Parametermarkierung
- `dbutils.widgets.remove("name")` – ein Widget entfernen
- `dbutils.widgets.removeAll()` – alle Widgets entfernen

Wichtige Einschränkung: In derselben Zelle, in der ein Widget entfernt wurde, kann kein neues Widget erstellt werden – dafür ist eine separate Zelle nötig.

Parametermarkierungen in SQL (ab Databricks Runtime 15.2)

Moderne Runtimes unterstützen Parametermarkierungen, die klar zwischen SQL-Befehl und eingegebenem Wert trennen und so vor SQL-Injection schützen:

```
SHOW TABLES IN IDENTIFIER(:database)
SELECT * FROM IDENTIFIER(:database || '.' || :table) WHERE col = :filter_value LIMIT 100
```

Für Objektbezeichner wie Datenbank-, Tabellen- oder Spaltennamen ist die `IDENTIFIER()` -Klausel erforderlich. Ab Databricks Runtime 18.0 werden Widgets auch direkt in DDL-Klauseln wie `LOCATION` unterstützt; ältere Runtimes (14.3–17.3 LTS) benötigen dafür `EXECUTE IMMEDIATE`.

Verhalten bei Wertänderung konfigurieren

Einstellung	Wirkung
Run Notebook	Führt das gesamte Notebook bei Wertänderung neu aus

Run Accessed Commands (Standard)	Führt nur die Zellen neu aus, die auf den geänderten Widget-Wert zugreifen (SQL-Zellen ausgenommen)
Do Nothing	Keine automatische Neuausführung

Widget-Panel anpassen

Nutzer können das Widget-Panel oben anheften, oberhalb der ersten Zelle positionieren und (mit "Can Manage"-Rechten) Reihenfolge und Größe konfigurieren oder auf Standard zurücksetzen.

Widgets in Dashboards

Wird ein Notebook mit Widgets zu einem Dashboard, erscheinen alle Widgets oben im Dashboard. Im Präsentationsmodus lassen sich Werte ändern; über den Update-Button wird das Notebook neu ausgeführt und die Diagramme aktualisiert.

Zusammenspiel mit %run

Beim Aufruf über `%run` werden standardmäßig die Standardwerte der Widgets im aufgerufenen Notebook verwendet. Auf Clustern (nicht auf SQL-Warehouses) lassen sich Werte überschreiben:

```
%run /pfad/zum/notebook $X="10" $Y="1"
```

Berechtigungen

Zum Erstellen/Bearbeiten von Widgets ist "Can Edit" nötig, für Layout-Änderungen "Can Manage".

Weitere Hinweise

Die vollständige API-Dokumentation lässt sich über `dbutils.widgets.help()` im Notebook abrufen. Ab Databricks Runtime 11.3 LTS wird zusätzlich `ipywidgets` als alternativer Ansatz unterstützt.

Quelle: <https://docs.databricks.com/aws/en/notebooks/widgets>

Stand: 2026-08-06